Team-Analysis

August 18, 2025

[1]:
```
# Questions for exploration
## 1. What are the team strengths?
## 2. What are the team strengths with 'Sabah' as part of team?
## 3. How does 'Sabah' result compare to the control (null) for Lead␣
 ↪Biostatistician role?

# Summary
## * The dataset itself needs deeper curation to robustly address questions
## * Estimated time needed to evaluate for precision and accuracy: ~1 week (vs␣
 ↪actual ~3 hrs)
## * Analysis primarily serves as a glimpse into capabilities and approach as a␣
 ↪team member
## * Hypothetically, this could be done as a team annually for determining␣
 ↪needs and direction
```

[2]:
```
# Upload team data
## 3 datasets: Current (B), Team with Sabah (A), Team with randomized control␣
 ↪(C)
## Note: Dataset based solely on profiles, likely needs follow-up for accurate␣
 ↪representation
```

[3]:
```
# Load relevant packages
suppressPackageStartupMessages({
  library(dplyr)      # v1.1.4 data manipulation
  library(purrr)      # v1.1.0 functional programming
  library(tidyr)      # v1.3.1 data tidying
  library(tibble)     # v3.3.0 clean output formatting
})

# Data_all <- read_csv("data.csv")
Data_all <- read.csv("Data.csv") %>%
  as_tibble()
cat("\nDataset A: Current team with Sabah (5 members)\n")
Data_all

Data_current <- Data_all %>%
  filter(Name != "Sabah")
```

```r
cat("\nDataset B: Current team (4 members)\n")
Data_current

## Dataset C: Current team with null control (5 members)
generate_null_row <- function(df) {
  # Helper function to calculate probabilities for multi-value columns
  calc_probs <- function(col, sep = ", ", min_count = 1) {
    df %>%
      separate_rows({{col}}, sep = sep) %>%
      filter(!is.na({{col}})) %>%
      count({{col}}) %>%
      mutate(prob = n / sum(n)) %>%
      filter(n >= min_count)
  }
  # Calculate probabilities for each column type
  probs <- list(
    Time_Zone = count(df, Time_Zone, wt = NULL, name = "n") %>% mutate(prob = n␣
↪/ sum(n)),
    Languages = calc_probs(Languages, min_count = 1),
    Education = calc_probs(Education, min_count = 1),
    Programming = calc_probs(Programming, min_count = 1),
    Statistical_Expertise = calc_probs(Statistical_Expertise, min_count = 1),
    Regulatory_Expertise = calc_probs(Regulatory_Expertise, min_count = 1)
  )
  # Sample values based on calculated probabilities
  tibble(
    Name = "Null_Control",
    Time_Zone = sample(probs$Time_Zone$Time_Zone, 1, prob =␣
↪probs$Time_Zone$prob),
    Languages = probs$Languages %>%
      slice_sample(n = 2, weight_by = prob) %>%
      pull(Languages) %>%
      paste(collapse = ", "),
    Education = probs$Education %>%
      slice_sample(n = 2, weight_by = prob) %>%
      pull(Education) %>%
      paste(collapse = ", "),
    Programming = probs$Programming %>%
      slice_sample(n = 4, weight_by = prob) %>%
      pull(Programming) %>%
      paste(collapse = ", "),
    Statistical_Expertise = probs$Statistical_Expertise %>%
      slice_sample(n = 3, weight_by = prob) %>%
      pull(Statistical_Expertise) %>%
      paste(collapse = ", "),
    Regulatory_Expertise = if_else(
      runif(1) > 0.3,
```

```
      probs$Regulatory_Expertise %>%
        slice_sample(n = 2, weight_by = prob) %>%
        pull(Regulatory_Expertise) %>%
        paste(collapse = ", "),
      NA_character_
    )
  )
}
# Generate null row using modal values from simulations
set.seed(123)
null_row_final <- map_dfr(1:100, ~generate_null_row(Data_current)) %>%
  summarise(across(everything(), ~names(sort(table(.), decreasing = TRUE)[1])))
# Create Dataset C by combining current data with the null row
Data_null <- bind_rows(Data_current, null_row_final)

cat("\nDataset C: Current team with null control (5 members)\n")
Data_null
```

```
Dataset A: Current team with Sabah (5 members)
```

A tibble: 5 × 7

Name <chr>	Time_Zone <chr>	Languages <chr>	Education <chr>
Carlos	PDT	Chinese, English, German, Japanese, Malay, Spanish	Biology, Biosta
Eugenia	EDT	English, Italian, Spanish	Biostatistics, E
Fefemwole	PDT	English, Ewe, Twi	Data_Science,
Karin	CDT	English	Linguistics, Ma
Sabah	MDT	English, Hindi, Portuguese, Spanish, Urdu	Biology, Bioch

```
Dataset B: Current team (4 members)
```

A tibble: 4 × 7

Name <chr>	Time_Zone <chr>	Languages <chr>	Education <chr>
Carlos	PDT	Chinese, English, German, Japanese, Malay, Spanish	Biology, Biosta
Eugenia	EDT	English, Italian, Spanish	Biostatistics, E
Fefemwole	PDT	English, Ewe, Twi	Data_Science,
Karin	CDT	English	Linguistics, Ma

```
Dataset C: Current team with null control (5 members)
```

A tibble: 5 × 7

Name <chr>	Time_Zone <chr>	Languages <chr>	Education <chr>
Carlos	PDT	Chinese, English, German, Japanese, Malay, Spanish	Biology, Bi
Eugenia	EDT	English, Italian, Spanish	Biostatistic
Fefemwole	PDT	English, Ewe, Twi	Data_Scier
Karin	CDT	English	Linguistics,
Null_Control	PDT	English, Twi	Statistics, E

[4]:
```
# Power Analyses: Power in the data, vs 'n' we need to address the questions
## Current power is ~10-11%, need minimum of 64 people to evaluate questions at␣
 ↪80% power
```

[5]:
```
# Load relevant packages
suppressPackageStartupMessages({
  library(dplyr)      # v1.1.4 data manipulation
  library(purrr)      # v1.1.0 functional programming
  library(tibble)     # v3.3.0 clean output formatting
  library(pwr)        # v1.3.0 power calculations
  library(scales)     # v1.4.0 percentage formatting
})

# Constants for power analysis
EFFECT_SIZE <- 0.5      # Medium effect size (Cohen's d)
ALPHA_LEVEL <- 0.05     # Default significance level (5%)
TARGET_POWER <- 0.8     # Default desired power (80%)

# Function to analyze power for a given scenario
analyze_power <- function(n1, n2, scenario_name) {
  # Calculate harmonic mean of sample sizes for unequal groups
  harmonic_n <- 2 / (1/n1 + 1/n2)
  # Calculate current power
  current <- pwr.t.test(
    d = EFFECT_SIZE,
    sig.level = ALPHA_LEVEL,
    n = harmonic_n,
    alternative = "two.sided",
    power = NULL
  )
  # Calculate required sample size per group for target power
  required <- pwr.t.test(
    d = EFFECT_SIZE,
    sig.level = ALPHA_LEVEL,
    power = TARGET_POWER,
    alternative = "two.sided",
    n = NULL
  )
  # Return as tibble row
  tibble(
    Scenario = scenario_name,
    Current_Power = percent(current$power, accuracy = 0.1),
    Current_n1 = n1,
    Current_n2 = n2,
    Required_n_per_group = ceiling(required$n),
    Effect_Size = EFFECT_SIZE,
    Significance_Level = ALPHA_LEVEL
```

```
  )
}

# Define scenarios to analyze as a list of lists
scenarios <- list(
  list(n1 = 4, n2 = 5, name = "Current (4) vs Team with Sabah (5)"),
  list(n1 = 4, n2 = 5, name = "Current (4) vs Team with Control (5)"),
  list(n1 = 5, n2 = 5, name = "Team with Sabah (5) vs Team with Control (5)")
)

# Run analysis for all scenarios
results <- map_dfr(scenarios, ~ {
  analyze_power(n1 = .x$n1, n2 = .x$n2, scenario_name = .x$name)
})

# Format and display results
results %>%
  select(Scenario, Current_Power, Current_n1, Current_n2, Required_n_per_group)␣
 ↪%>%
  print()
```

```
# A tibble: 3 × 5
  Scenario              Current_Power Current_n1 Current_n2 Required_n_per_group
  <chr>                 <chr>              <dbl>      <dbl>                <dbl>
1 Current (4) vs Team … 9.9%                   4          5
64
2 Current (4) vs Team … 9.9%                   4          5
64
3 Team with Sabah (5) … 10.8%                  5          5
64
```

[6]:
```
# Fisher's: Comparisons of datasets for 'Programming' and 'Regulatory Expertise'
## Unique results with 'Sabah' present in team, though ~mute as dataset not␣
 ↪robust
```

[7]:
```
# Load relevant packages
suppressPackageStartupMessages({
  library(dplyr)       # v1.1.4 data manipulation
  library(tidyr)       # v1.3.1 data tidying
  library(purrr)       # v1.1.0 functional programming
  library(ggplot2)     # v3.5.2 visualization
  library(ggvenn)      # v0.1.10 venn diagrams
  library(stringr)     # v1.5.1 string manipulation
  library(broom)       # v1.0.9 statistical test tidying
  library(patchwork)   # v1.3.1 combining plots
```

```
})

# Function to analyze a specific variable
analyze_variable <- function(variable_name) {
  ## Prepare data
  extract_items <- function(df, dataset_name) {
    df %>%
      select(Name, !!sym(variable_name)) %>%
      separate_rows(!!sym(variable_name), sep = ", ") %>%
      filter(!is.na(!!sym(variable_name)) & !!sym(variable_name) != "") %>%
      mutate(Dataset = dataset_name) %>%
      distinct(!!sym(variable_name), Dataset)
  }
  items_combined <- bind_rows(
    extract_items(Data_current, "Current (4)"),
    extract_items(Data_all, "With Sabah (5)"),
    extract_items(Data_null, "With Control (5)")
  )
  ## Statistical comparisons
  presence_matrix <- items_combined %>%
    mutate(Present = 1) %>%
    pivot_wider(names_from = Dataset, values_from = Present, values_fill = 0)␣
↪%>%
    as.data.frame()
  pairwise_tests <- combn(c("Current (4)", "With Sabah (5)", "With Control␣
↪(5)"), 2, simplify = FALSE) %>%
    map_dfr(function(pair) {
      test_data <- presence_matrix[, pair]
      if (ncol(test_data) == 2 && nrow(test_data) > 0) {
        test_result <- tryCatch(
          fisher.test(test_data),
          error = function(e) list(p.value = NA, estimate = NA, conf.int =␣
↪c(NA, NA))
        )
        tibble(
          Comparison = paste(pair, collapse = " vs "),
          p_value = test_result$p.value,
          odds_ratio = test_result$estimate,
          conf_low = test_result$conf.int[1],
          conf_high = test_result$conf.int[2]
        )
      } else {
        tibble(
          Comparison = paste(pair, collapse = " vs "),
          p_value = NA,
          odds_ratio = NA,
          conf_low = NA,
```

```
        conf_high = NA
      )
    }
  }) %>%
  mutate(p_adj = p.adjust(p_value, method = "BH")) %>%
  arrange(p_value)
## Venn diagram
venn_data <- list(
  `Current Team (4)` = unique(filter(items_combined, Dataset == "Current (4)")[[variable_name]]),
  `With Sabah (5)` = unique(filter(items_combined, Dataset == "With Sabah (5)")[[variable_name]]),
  `With Control (5)` = unique(filter(items_combined, Dataset == "With Control (5)")[[variable_name]])
)
## Create venn diagram
venn_plot <- ggvenn(
  venn_data,
  fill_color = c("#0073C2FF", "#EFC000FF", "#868686FF"),
  stroke_size = 0.5,
  set_name_size = 4,
  text_size = 3,
  show_percentage = FALSE
) +
  labs(title = paste("Overlap of", variable_name)) +
  theme_void() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))
## Detailed breakdown
get_intersection_details <- function(sets) {
  combinations <- list(
    list(name = "Current only", sets = c(1)),
    list(name = "Sabah only", sets = c(2)),
    list(name = "Control only", sets = c(3)),
    list(name = "Current & Sabah", sets = c(1,2)),
    list(name = "Current & Control", sets = c(1,3)),
    list(name = "Sabah & Control", sets = c(2,3)),
    list(name = "All three", sets = c(1,2,3))
  )
  intersection_details <- map_dfr(combinations, function(comb) {
    if (length(comb$sets) == 1) {
      items <- setdiff(sets[[comb$sets[1]]], unique(unlist(sets[-comb$sets[1]])))
    } else if (length(comb$sets) == 2) {
      items <- intersect(sets[[comb$sets[1]]], sets[[comb$sets[2]]])
      items <- setdiff(items, reduce(sets, intersect))
    } else {
      items <- reduce(sets, intersect)
```

```
      }
      if (length(items) > 0) {
        tibble(
          Section = comb$name,
          Items = paste(items, collapse = ", "),
          Count = length(items)
        )
      }
    })
    return(intersection_details)
  }
  intersection_details <- get_intersection_details(venn_data)
  return(list(
    variable = variable_name,
    plot = venn_plot,
    stats = pairwise_tests,
    details = intersection_details
  ))
}

# Analyze both variables
programming_results <- analyze_variable("Programming")
regulatory_results <- analyze_variable("Regulatory_Expertise")

# Display side-by-side Venn diagrams
combined_plots <- programming_results$plot + regulatory_results$plot +
  plot_layout(ncol = 2) +
  plot_annotation(title = "Comparison of Team Attributes",
                  subtitle = "Programming Skills and Regulatory Expertise",
                  theme = theme(plot.title = element_text(hjust = 0.5, face =␣
 ↪"bold"),
                                plot.subtitle = element_text(hjust = 0.5)))

print(combined_plots)

# Print statistical results
cat("\n=== Programming Skills ===\n")
cat("\nStatistical Comparisons:\n")
print(programming_results$stats, n = Inf)
cat("\nDetailed Breakdown:\n")
print(programming_results$details %>% arrange(desc(Count)), n = Inf)

cat("\n\n=== Regulatory Expertise ===\n")
cat("\nStatistical Comparisons:\n")
print(regulatory_results$stats, n = Inf)
cat("\nDetailed Breakdown:\n")
print(regulatory_results$details %>% arrange(desc(Count)), n = Inf)
```

```
Warning message:
"package ‘ggvenn’ was built under R version 4.1.2"


=== Programming Skills ===

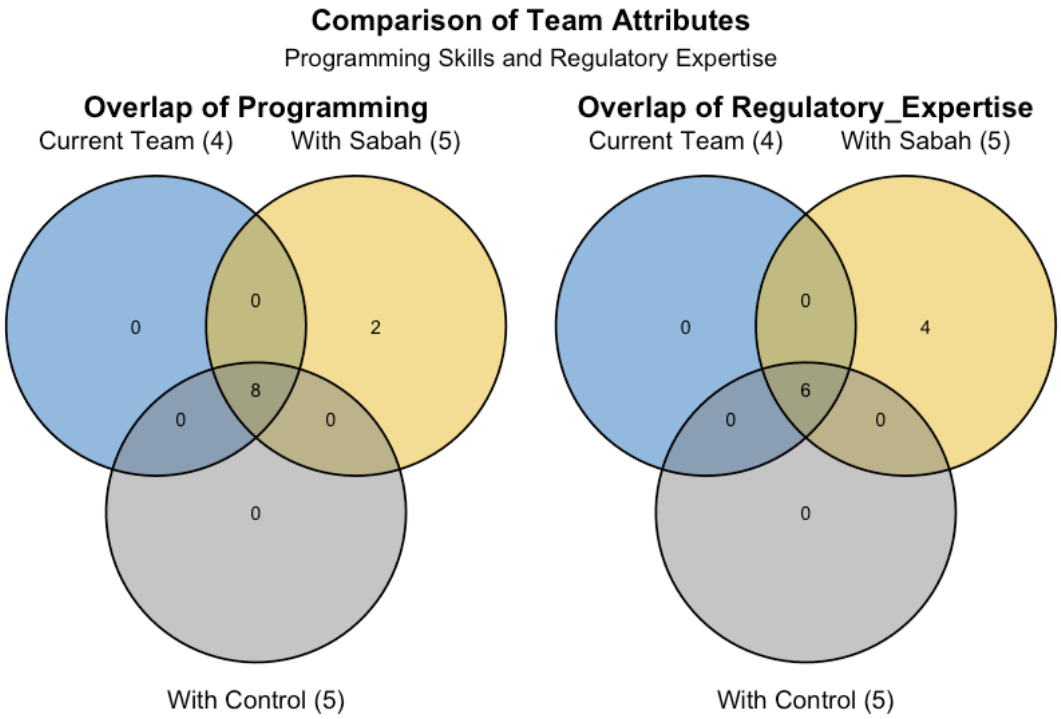
Statistical Comparisons:
# A tibble: 3 × 3
  Comparison                          p_value p_adj
  <chr>
<dbl> <dbl>
1 Current (4) vs With Sabah (5)         1.000     1
2 With Sabah (5) vs With Control (5)    1.000     1
3 Current (4) vs With Control (5)       1         1

Detailed Breakdown:
# A tibble: 2 × 3
  Section    Items                                                    Count
  <chr>      <chr>
<int>
1 All three  Matlab, R, Python, SQL, R_Shiny, QlikSense, Mplus, SPSS
8
2 Sabah only Bash, HTML
2


=== Regulatory Expertise ===

Statistical Comparisons:
# A tibble: 3 × 3
  Comparison                          p_value p_adj
  <chr>
<dbl> <dbl>
1 Current (4) vs With Sabah (5)             1     1
2 Current (4) vs With Control (5)           1     1
3 With Sabah (5) vs With Control (5)        1     1

Detailed Breakdown:
# A tibble: 2 × 3
  Section    Items                             Count
  <chr>      <chr>
<int>
1 All three  CAPA, CLIA, FDA, CAP, CLSI, GCP     6
2 Sabah only HC, MHRA, TGA, WHO                  4
```


Comparison of Team Attributes
Programming Skills and Regulatory Expertise
Overlap of Programming
Current Team (4)
With Sabah (5)
0
0
2
8
0
0
0
With Control (5)
Overlap of Regulatory_Expertise
Current Team (4)
With Sabah (5)
0
0
4
6
0
0
0
With Control (5)